

8/7/26

Cloud Computing

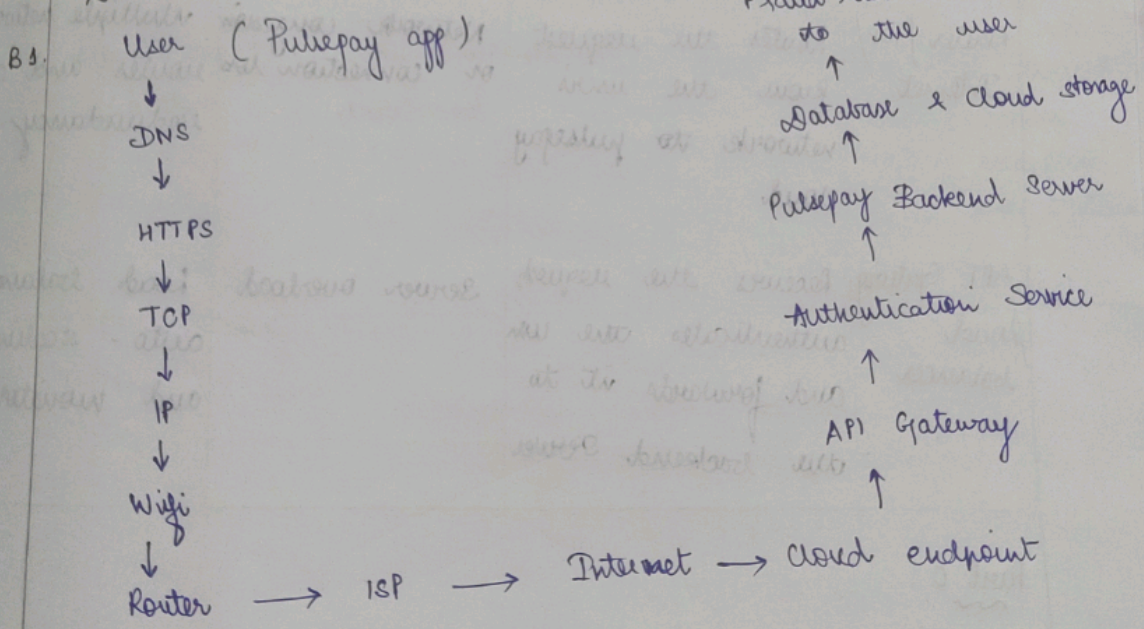
Part A:

A1. Project title - Pulse pay cloud fake Payment Screenshot Detector with merchant Verification Workflow

Main user action - Upload a payment screenshot to verify whether the transaction is genuine or potentially fraudulent.

A2. Upload this payment screenshot to the Pulsepay cloud, verify the transaction, calculate the fraud risk score and show the verification risk.

Part B:



B2.	Layer / Component	what it does in your app request	Possible failure	cloud-side protection
	DNS	Converts the Pulsepay domain name into cloud server IP add.	DNS lookup failure	Backup DNS & DNS caching
	HTTPS/TLS	encrypts the payment screenshot and user data during Trans - mission	Invalid or expired certificate	TLS certificate and HTTPS enforcement
	TCP/IP	Transfers the request packets reliably between the phone and the cloud server	Packet loss or Timeout	TCP retransmission and error checking.
	Router / ISP / Internet	Routes the request from the user's network to pulsepay cloud	Network congestion or connection loss	Multiple network routes and cloud redundancy.
	API Gateway / Load balancer	Receives the request, authenticates the user and forwards it to the backend server	Server overload	Load balancing, auto - scaling and monitoring

Part C:

Cloud need	Your component choice	Service model	Justification
User identity / login	Authentication Service (Firebase authentication)	SaaS	Authenticates merchant securely before accessing Pulsepay Service

2026.07.08 10:55

side protection
backup and
caching

certificates
HTTPS
element

retransmission
error
ng.

the network
and cloud
ancy.

balancing.
caching
monitoring

tion
merchants

re
epay

App Backend/
Business Logic

Database/
records

File/Image
Storage

Monitoring/
analytics

Cloud API Server
(Backend)

Cloud database
(Firestore / MySQL)

Cloud storage

Cloud monitoring
dashboard

PaaS

PaaS

PaaS

SaaS

Process payment
screenshots, performs
fraud analysis
and returns the
verification result.

Stores merchant
details, verification
history and fraud
risk scores

Securely stores
uploaded payment
screenshots for
verification and
future reference

Monitors request
count, server
health, response
time and system
performance

pro

Part D:

Field	Answer
Endpoint URL path	/api/pulapay/verify-payment
HTTP method	POST
Request data	Merchant ID, payment screenshot, transaction ID, date and time
Response data	Verification status, fraud risk score, message, transaction ID.
Authentication Rule	The merchant must log in using a secure authentication token. All requests are sent through HTTPS/TLS via API gateway
Error Case	If the screenshot is missing, unreadable or the server cannot process the request, return HTTP 400 (Bad request) or HTTP 500 (Internal server error) with appropriate error message.

Part E:

E₁) Total request = 7,200
Time = 10 minutes

$$\Rightarrow \text{Peak requests per minute} = \frac{7200}{10} = 720 \text{ req/min}$$

E₂) 25% of 720 = 180

$$\text{Required capacity} = 720 + 180 = 900 \text{ req/min}$$

2026.07.08 10:55

E3. one cloud instance = 180 req/minute

Required instances = $\frac{900}{180} = 5$ instances

E4. Pulse pay uses auto-scaling to automatically add or remove cloud API server instances based on their number of payments during peak traffic.

E5. 1. API request count per minute

2. Average response latency (response time)

Part F:

Search/prompt used	Tool used	Useful finding	How I applied it
"why should APIs use HTTPS/TLS for secure data transfer?"	chatgpt/ Google	HTTPS/TLS encrypts data between the client and the cloud server, protecting it from interception during transmission	I used HTTPS/TLS in the Pulsepay API so uploaded payment screenshots and merchant data are securely transmitted to the cloud through API Gateway.